

BÁO CÁO TÓM TẮT BÀI TẬP LỚN

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

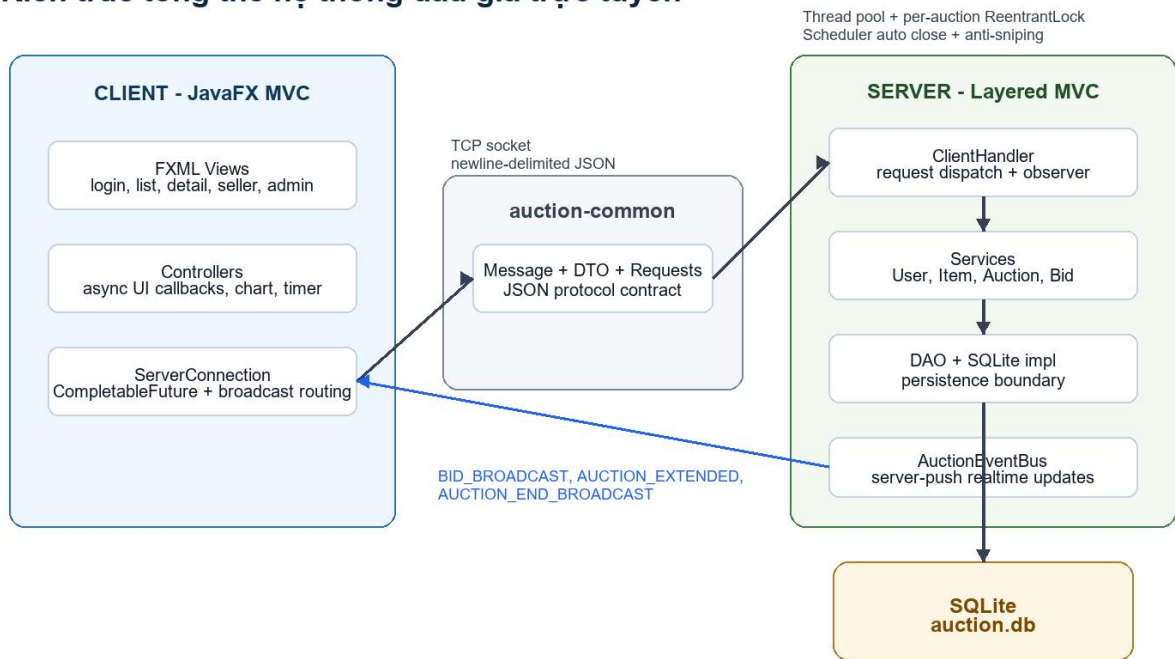
1. Mục tiêu và phạm vi thực hiện

Mục tiêu là xây dựng một hệ thống đấu giá trực tuyến dạng client-server, cho phép nhiều người dùng tham gia phiên đấu giá theo thời gian thực, theo thiết kế hướng đối tượng, xử lý đồng thời và kiến trúc phân tầng theo yêu cầu đề bài.

Phạm vi đã thực hiện gồm: quản lý tài khoản theo vai trò Bidder/Seller/Admin; quản lý sản phẩm và phiên đấu giá; đặt giá, auto-bidding, kết thúc phiên tự động; cập nhật realtime qua socket; giao diện JavaFX/FXML; lưu trữ SQLite; kiểm thử JUnit và CI GitHub Actions.

2. Kiến trúc tổng thể

Kiến trúc tổng thể hệ thống đấu giá trực tuyến



Hình 1. Kiến trúc tổng quát

Hệ thống chia thành ba module Maven. auction-client chứa giao diện JavaFX theo MVC: FXML định nghĩa view, controller xử lý thao tác người dùng, ServerConnection gửi request và nhận broadcast bất đồng bộ. auction-common định nghĩa hợp đồng dùng chung gồm Message, MessageType, DTO và payload request/response để hai phía serialize JSON thống nhất. auction-server nhận kết nối TCP, mỗi ClientHandler dispatch request sang service tương ứng, service thực thi nghiệp vụ và gọi DAO để thao tác SQLite.

Chỉ server truy cập database. Điều này giữ dữ liệu nhất quán và cho phép kiểm soát đồng thời tập trung. Với realtime, client gửi WATCH_AUCTION khi mở màn hình chi tiết; ClientHandler đăng ký vào AuctionEventBus. Khi có bid, gia hạn hoặc kết thúc phiên, server push broadcast đến các client đang theo dõi mà không cần polling.

3. Chức năng đạt được theo barem

Barem	Chức năng đạt được	Hướng giải quyết / lý do	Điểm
Thiết kế lớp và OOP	Có Entity, User/Bidder/Seller/Admin, Item/Electronics/Art/Vehicle, Auction, BidTransaction; sử dụng encapsulation, inheritance, polymorphism, abstraction theo yêu cầu đề bài.	Cách làm: xây dựng domain model trước, để Entity chứa thuộc tính chung; User được tách thành Bidder/Seller/Admin, Item được tách thành Electronics/Art/Vehicle. Lý do: phần nghiệp vụ không phụ thuộc SQL hay socket, thể hiện rõ kế thừa, đa hình và đóng gói; khi thêm vai trò hoặc loại sản phẩm mới chỉ cần mở rộng subclass/factory thay vì sửa toàn bộ luồng xử lý.	1.0 (các lớp chính) + 0.5 (OOP) = 1.5
Design pattern	Singleton (của DatabaseManager), Factory Method (tạo user và item theo đúng subclass), Observer, DAO (tương tác với database), MVC; Strategy qua comparator PriorityQueue của auto-bid.	Cách làm: DatabaseManager dùng Singleton để quản lý kết nối SQLite; UserFactory/ItemFactory tạo đúng subclass; DAO bao bọc thao tác lưu trữ; MVC tách giao diện JavaFX khỏi xử lý; AuctionEventBus/AuctionObserver đẩy sự kiện realtime; comparator của PriorityQueue đóng vai trò Strategy cho auto-bid. Lý do: mỗi pattern được dùng tại điểm mà tại đó nó sẽ giúp ích, phân tách trách nhiệm và dễ bảo trì hơn so với gộp SQL, UI và logic vào cùng lớp.	1.0
Người dùng và sản phẩm	Đăng ký, đăng nhập, phân quyền Bidder/Seller/Admin; seller tạo item, tạo/hủy phiên, đánh dấu paid; admin quản lý user.	Cách làm: UserService xử lý đăng ký, đăng nhập, kiểm tra role và trạng thái active; ItemService chỉ cho Seller tạo/quản lý sản phẩm; ClientHandler xác thực currentUser trước khi gọi service; DAO lưu người dùng, sản phẩm và trạng thái vào SQLite. Lý do: quyền truy cập được kiểm soát ở server nên client không thể tự ý vượt quyền, dữ liệu nhất quán giữa các phiên đăng nhập và admin có thể quản trị user tập trung. Nói chung, server là nơi kiểm soát và lưu trữ dữ liệu xác thực đúng, tránh việc client tự sửa mã, gửi tín hiệu giả lên server, client chỉ là nơi để giao tiếp với server.	1.0
Đấu giá và kết thúc phiên	Bid hợp lệ phải cao hơn giá hiện tại; cập nhật người dẫn đầu; scheduler tự đóng phiên và xác định FINISHED/CANCELED/PAID.	Cách làm: BidService kiểm tra trạng thái phiên, thời gian kết thúc, mức giá tối thiểu và người dẫn đầu trước khi lưu BidTransaction; AuctionService chuyển trạng thái OPEN/RUNNING/FINISHED/CANCELED/PAID, lên lịch đóng phiên	1.0

		và xác định winner khi hết hạn. Lý do: tách bid khỏi vòng đời auction giúp kiểm tra luật đặt giá rõ ràng và dễ dàng, tăng tính dễ bảo trì và phân tách trách nhiệm.	
Lỗi, ngoại lệ, concurrency	AuthException, AuctionException, BidException; per-auction ReentrantLock chống lost update/race condition.	Cách làm: dùng AuthException, AuctionException và BidException cho từng nhóm lỗi; mỗi auction có ReentrantLock trong BidService; các DAO SQLite dùng synchronized cho thao tác ghi; dữ liệu nhạy cảm như currentPrice/endTime được cập nhật trong vùng kiểm soát synchronized. Lý do: thông báo lỗi rõ cho client, đồng thời chống lost update/race condition khi nhiều bidder đặt giá vào cùng một phiên mà không khóa toàn bộ hệ thống (nếu cả hệ thống chỉ có một lock hay synchronized thì phiên này sẽ khóa của phiên kia, các phiên phải đợi nhau dùng chung khóa).	$1.0 (\text{xử lý lỗi}) + 1.0(\text{concurrency}) = 2.0$
Realtime update	Client watch auction nhận BID_BROADCAST, AUCTION_EXTENDED, AUCTION_END_BROADCAST qua socket.	Cách làm: khi mở chi tiết phiên, client gửi WATCH_AUCTION để ClientHandler đăng ký vào AuctionEventBus; khi có bid mới, gia hạn anti-sniping hoặc kết thúc phiên, server gửi BID_BROADCAST, AUCTION_EXTENDED, AUCTION_END_BROADCAST; client cập nhật UI qua Platform.runLater. Tất cả sử dụng TCP socket thay vì REST. Lý do: server push giúp màn hình chi tiết luôn đồng bộ gần thời gian thực, tránh polling liên tục gây trễ và tốn tài nguyên.	0.5
Kiến trúc, MVC, kiểm tra chất lượng	Client JavaFX + FXML; server Handler -> Service -> DAO; Maven multi-module; JUnit/Mockito; GitHub Actions chạy test.	Cách làm: chia Maven multi-module thành auction-common, auction-server và auction-client; common chứa DTO/protocol dùng chung, server đi theo Handler -> Service -> DAO, client đi theo JavaFX/FXML controller; test JUnit/Mockito và GitHub Actions kiểm tra tự động. Lý do: ranh giới module rõ giúp build độc lập, giảm phụ thuộc, cho phép chạy local trên nhiều máy trong cùng một mạng LAN (chi tiết cách chạy xem trong README)	$0.5(\text{kiến trúc client-server}) + 0.5(\text{MVC}) + 0.5(\text{Maven}) + 0.5(\text{Unit Test}) + 0.5(\text{CI/CD cơ bản}) = 2.5$
Nâng cao	Auto-bidding maxBid/increment; anti-sniping 30s + 60s; line chart lịch sử giá realtime.	Cách làm: auto-bidding lưu maxBid/increment, dùng PriorityQueue ưu tiên maxBid	$0.5(\text{mỗi chức năng}) * 3 = 1.5$

		cao hơn và registeredAt sớm hơn; anti-sniping gia hạn 60 giây khi bid đến trong 30 giây cuối; biểu đồ line chart đọc lịch sử BidTransaction và cập nhật theo broadcast.	
--	--	---	--

4. Giải thích thêm về các hướng giải quyết kỹ thuật

- TCP socket thay cho REST vì nghiệp vụ yêu cầu server push bid mới ngay lập tức đến tất cả client đang xem phiên. Cụ thể, REST là request-response, nếu client cần cập nhật giá theo thời gian thực thì phải gửi yêu cầu kiểm tra bid mới liên tục đến server, điều này thì có thể gây lag, lãng phí tài nguyên. Ngoài ra, ví dụ trong trường hợp hệ thống để mặc định 1 giây sẽ kiểm tra bid mới một lần, người dùng A đặt bid trong 1 giây đó nhưng trong khoảng thời gian ấy đã có nhiều bid mới cao hơn mức giá mà người dùng A định đặt thì bid của người dùng A sẽ lỗi dù trên màn hình mức giá định đặt vẫn cao hơn mức giá hiện tại của item.
- SQLite được chọn vì triển khai gọn, một file dữ liệu, không cần chạy riêng một server database, không cần cấu hình gì thêm; DAO synchronized và service lock bảo vệ điểm ghi nhạy cảm.
- Mỗi phiên đấu giá có một ReentrantLock trong BidService giúp tuần tự hóa bid trong cùng một phiên nhưng vẫn cho phép các phiên khác chạy song song. Cụ thể, một khóa synchronized toàn hệ thống sẽ tuần tự hóa toàn bộ bid trên toàn bộ phiên, bid trong phiên #1 sẽ chặn bid trong phiên #2. Thay vào đó ta sử dụng ConcurrentHashMap lưu giữ mỗi phiên cùng với lock của riêng nó, kết quả đạt được là bid trên các phiên khác nhau vẫn có thể chạy song song và các bid trong cùng phiên vẫn tránh được lost-update / race condition.
- PriorityQueue cho auto-bid giúp chọn người có maxBid cao nhất, nếu bằng nhau ưu tiên người đăng ký trước; mỗi auto-bid được lưu như BidTransaction.
- ScheduledExecutorService trong AuctionService tự đóng phiên theo endTime; anti-sniping gia hạn khi bid đến trong 30 giây cuối.